

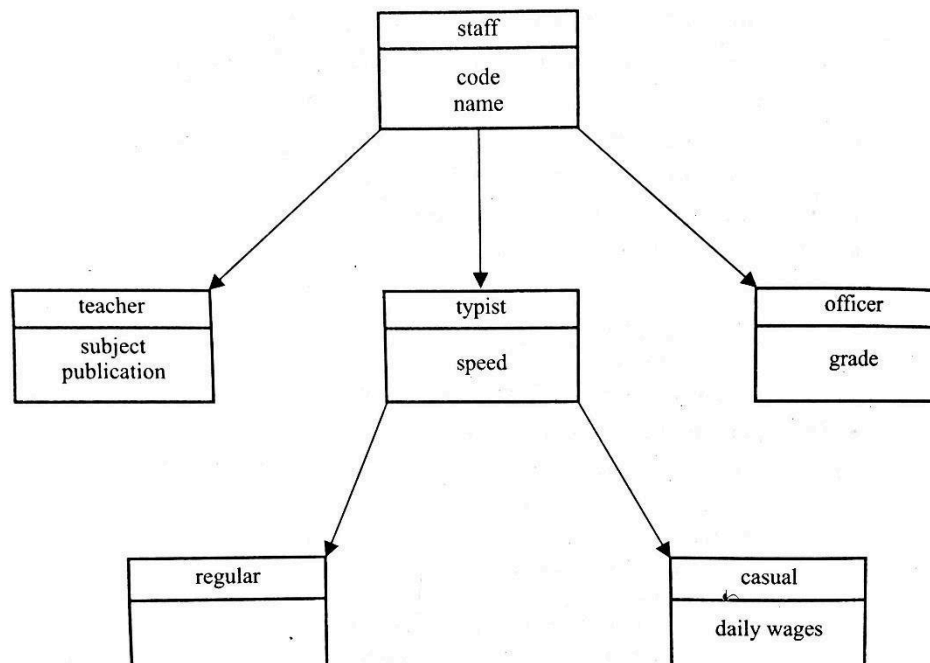
1. Design a class to represent a bank account. Include following members:

Data members: (a) Name of the depositors (b) Account Number (c) Account Type (Savings and Current) (d) Balance amount

Methods: (a) Constructor to assign initial value (b) To deposit an amount (c) To withdraw an amount after checking balance. (d) To display the name and balance.

Write a menu driven java program to solve the problem.

2. An educational institution wishes to maintain a database of its employees. The database is divided into a number of classes whose hierarchical relationships are shown below. The figure also shows the minimum information required for each class. Specify all the classes and define methods to create the database and retrieve individual information as and when required.



3. Vector with String Elements

Objective: Perform operations with Vector to store and manipulate strings.

Task:

1. Create a Vector to store strings.
2. Add the names of fruits, countries, colors etc. to the vector.
3. Print the vector contents.
4. Search for a specific item in the vector and print item and its index.
5. Replace an item in the vector with another item taking from user.
6. Sort the vector both in ascending and descending order depending on the choice of user.
7. Print the updated vector.

4. The daily maximum temperatures recorded in 10 cities during the month of January, 2025 (31 days) have been tabulated as follows:

Day	City1	City2	City3	City10
			...	
1				
2				
3				
.				
.				
.				
31				

Write a program to read the table elements into a two-dimensional array temperature and to find the city and day corresponding to (a) highest temperature and (b) the lowest temperature.

For programming simplicity, take temperature for 5 days only and take 5 cities only.

5. Modifying Objects in a Vector

Objective: Modify the fields of objects stored in a Vector.

Task:

1. Create a Student class with fields name, id, and grade.
2. Create a Vector to store Student objects.
3. Add some Student objects to the vector.
4. Change any field depending on the choice of the user.
5. Display the updated vector.

6. An election is contested by 5 candidates. The candidates are numbered 1 to 5 and the voting is done by marking the candidate number on the ballot paper. Write a menu driven program to read the ballots and count the votes cast for each candidate using an array variable count. In case, a number read is outside the range 1 to 5, the ballot should be considered as a 'spoilt ballot' and the program should also count the number of spoilt ballots.

7. Searching for an Object and Removing that object in a Vector

Objective: Learn how to search for an object in a Vector based on a custom condition.

Task:

1. Create a Movie class with fields title, director, and releaseYear.

2. Create a Vector to store Movie objects.
3. Add multiple Movie objects to the vector.
4. Search for a movie by its title and display its details.
5. Ask for deletion and if the answer is yes, delete the object from the vector.

8. The annual examination results of 100 students are tabulated as follows:

Roll No.	Subject-1	Subject-2	Subject-3
.			
.			
.			
.			

Write a program to read the data and determine the following:

- (a) Total marks obtained by each student.
 - (b) The highest marks in each subject and the Roll No. of the student who secured it.
 - (c) The student who obtained the highest total marks.
-

9. Sorting Objects in a Vector

Objective: Learn how to sort objects in a Vector based on a custom property.

Task:

1. Create a Employee class with fields name, age, and salary.
 2. Create a Vector to store Employee objects.
 3. Add multiple Employee objects to the vector.
 4. Sort the vector based on the salary of employees in both ascending order and descending depending on the choice.
 5. Display the sorted vector.
-

10. Assume that a bank maintains two kinds of account for its customers - savings and current.

The savings account provides compound interest and withdrawal facilities but no checkbook facility.

The current account holders provides checkbook facility but no interest. The current account holders should also maintain a minimum balance and if the balance falls below this level, a service charge is imposed.

Create a class Account that stores customer name, account number and type of account. From this derive the classes Curr-acct and Sav-acct to make them more specific to their requirements. Include necessary methods in order to achieve the following tasks:

- (a) Accept deposit from a customer and update the balance.
- (b) Display the balance.
- (c) Compute and deposit interest.
- (d) Permit withdrawal and update the balance.

(e) Check for the minimum balance, impose penalty, if necessary and update the balance.

Write a menu driven java program to solve the problem.

11. Write a menu driven program that will accept a shopping list of five items and will store in a vector. Perform following operation on the vector:

- (a) To delete an item in the list.
 - (b) To add an item at a specified location in the list.
 - (c) To add an item at the end of the list.
 - (d) To print the contents of the vector.
-

12. Write a program that will perform following operations:

- (a) read a string and will count number of occurrences of each word.
 - (b) read a string and will arrange the letters in both ascending and descending order depending on the choice of user. For example, if the word is STRING, the result will be GINRST or TSRNIG.
-

13 Write a Java program having an array of 10 elements, implement two threads for sorting the array, one using bubble sort and the other using index sort, display them from both the thread and check which thread to complete first.

14. Bank Account Transaction System (Polymorphism with Different Account Types)

Objective: Use polymorphism to model different types of bank accounts with different transaction types.

Task:

1. Create an abstract class BankAccount with a balance field and a method performTransaction().
2. Create subclasses CheckingAccount and SavingsAccount.
 - CheckingAccount allows for withdrawal and deposit transactions.
 - SavingsAccount allows only deposits, with a method for calculating interest.
3. In the main class, create an array of BankAccount and perform transactions using polymorphism.

15. Product Discount System (Polymorphism with Different Discount Strategies)

Objective: Demonstrate polymorphism by implementing different discount strategies.

Task:

1. Create a Product class with fields name and price, and a method applyDiscount().
2. Create three subclasses Electronics, Clothing, and Food that implement different discount strategies for each product category.

3. The Electronics class should provide a 10% discount, Clothing should provide a 20% discount, and Food should provide a 5% discount.

In the main class, create an array of Product objects and apply the respective discounts using polymorphism.

16. Employee Benefits System (Polymorphism with Different Benefits)

Objective: Demonstrate polymorphism through overriding and handling different types of employee benefits.

Task:

1. Create a superclass Employee with fields name and salary, and a method calculateBenefits().
2. Create subclasses FullTimeEmployee and PartTimeEmployee. Each subclass overrides the calculateBenefits() method.
3. The FullTimeEmployee class should provide benefits like health insurance, while the PartTimeEmployee class should provide a limited set of benefits (e.g., hourly benefits or no benefits).
4. In the main class, create an array of Employee objects and calculate benefits for each using polymorphism.

17. Bank Account System

Objective: Use polymorphism to implement different types of bank accounts.

Task:

1. Create a superclass BankAccount with common fields such as accountNumber and balance, and a method withdraw().
2. Create two subclasses SavingsAccount and CheckingAccount that have different rules for withdrawing money.
3. Use polymorphism to perform withdrawals and print the result.

18. Vehicle Class Hierarchy

Objective: Use polymorphism with method overriding to simulate different types of vehicles.

Task:

1. Create a superclass Vehicle with a method start() that prints a generic message.
2. Create two subclasses Car and Bike that override the start() method with their own implementations.
3. In the main class, create an array of Vehicle type and call the start() method on each object using polymorphism.

19. Employee Salary Calculation

Objective: Implement polymorphism using inheritance.

Task:

1. Create a superclass Employee with fields like name and salary, and a method calculateSalary().
2. Create two subclasses FullTimeEmployee and PartTimeEmployee that override calculateSalary() to implement different salary calculation methods.
3. In the main class, create an array of Employee type and add both FullTimeEmployee and PartTimeEmployee objects to it. Use polymorphism to calculate and display the salary for each employee.

20. Write a Java program to demonstrate autoboxing (converting a primitive type to a wrapper object) and unboxing (converting a wrapper object back to a primitive).

21. Design a "Student" Class with fields like "name", "rolino" and "address". Write a derived class called "MCA_Student" with fields like department, semester, etc. Write proper constructors for both classes with a proper display() method in the derived class to display the details of MCA_Student. Use a Demo class to demonstrate the above.

22. Write a Tree (super class) to represent the trees in a farm. A tree has the following attributes

- a. Tree Code no
- b. Height
- c. Base
- d Width
- e. AmtSpnt

Write a method, annual Uplate(), which updates the height, base, width, and amount spent so far on this tree, and a method, display Tree() which will display the details of a Tree.

Define a Mango (sub class) which is a derved class of the class Tree. In addition to the attributes of the Tree class, the Mango class has a yield attribute. Overnde the display Tree() and annualUpdata() methods suitably

23. In a billing system, define a super class with inventory items as attributes such as itemID, description, unit, and price. Define this as an item class and define the necessary constructors and display methods.

In the above design, create a B_item class (as sub class), when a item is sold, a bill is produced. The bill may be for several items bought. In the bill, each item is shown as a row. Define this as a class B_item (Bill Item), and add the necessary attributes, and define the necessary constructor and display method.

24. Write a program that has a super class called "Box" with instance variables width, height and depth and the corresponding method to display them all. There will be a subclass called "BoxWeight" which has an instance variable called density and a method that displays the weight of the box. Write the corresponding main method to demonstrate proper inheritance.

25. Create a generic base class called "Vehicle" that stores the number of wheels and speed. Create the following derived classes listed below:

a. "Car" that inherits "Vehicle" and also stores the number of passengers.

b. Truck that inherits "Vehicle" and also stores the load limit

Write a main() function to create objects of these classes and display all the information about "Car" and "Truck" Also, compare the speeds of the two vehicles, "Car" and "Truck", and display "faster" or "slower" if "Car" is faster or slower than "Truck"

26. Develop an abstract class "2DFigure" that will have two variables, namely, color and weight; it should have constructors for setting the default color as "white" and default weight as 1.0. The class should have methods getColor() and getWeight() to return the color and weight values to the caller. The class should have two abstract methods, findArea() and findCircumferences().

a. Write Subclasses for "2DFigure" called "Triangle" and "Rectangle" that will be able to calculate the area and circumference of the respective types.

b. Using the references in "2DFigure" to call the overridden method of subclasses of respective types to demonstrate run-time polymorphism.

27. Give an example where an interface can be used to support multiple inheritance. Develop a standalone Java program for the example.